Computer Organization and Architecture
(Code: CS256)

Lecture-19: Cache Memory Design

Course Faculty

Dr. B. R. Bhowmik

Assistant Professor

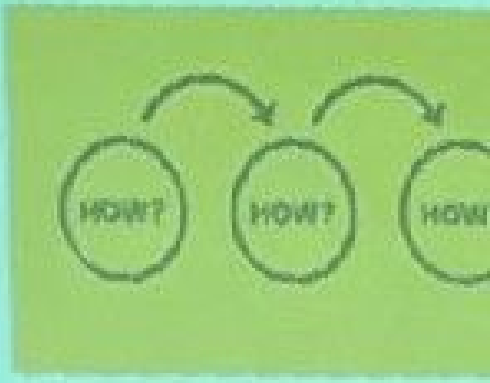
Dept. of CSE

Website: https://sites.google.com/view/biswajitrbhowmik
https://sites.google.com/view/vicayana-gaveshana

Course Email: sangsangstha@gmail.com

Why Memory Hierarchy?

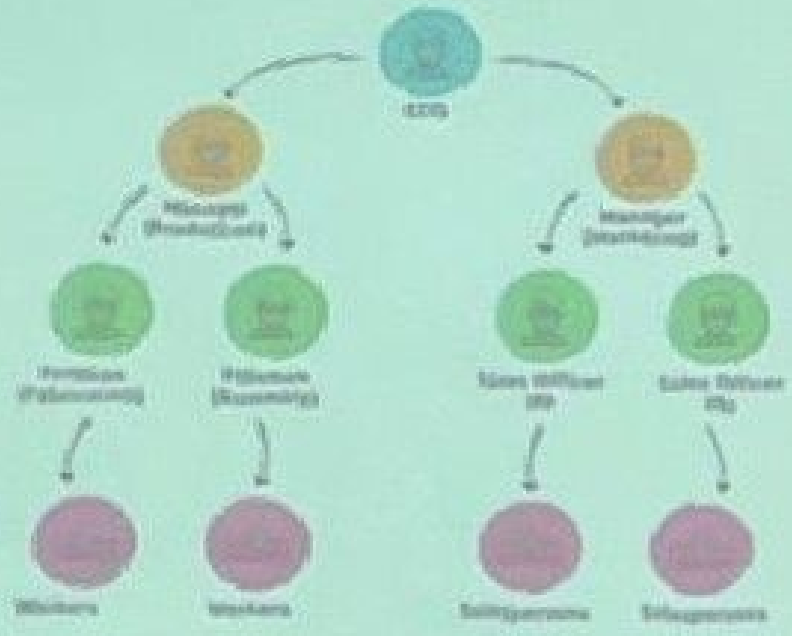
- Design constraints on a computer's memory can be summed up by three questions:
  - **How much,**
  - **How fast,**
  - **How expensive**

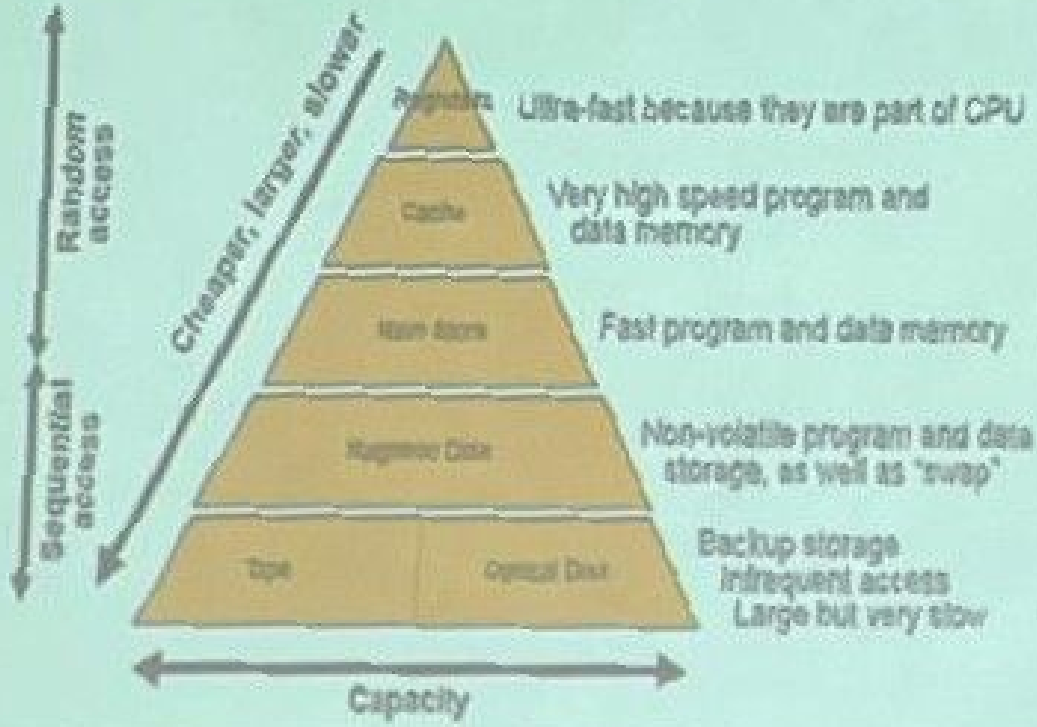


- There is a trade-off among capacity, access time, and cost
  - *Policy-1:* Faster access time, greater cost per bit
  - *Policy-2:* Greater capacity, smaller cost per bit
  - *Policy-3:* Greater capacity, slower access time

Memory Hierarchy

- A hierarchy is an arrangement of items that are placed in a top-down manner according to their activities, functionalities, etc.
- The way out of the memory dilemma is not to rely on a single memory component or technology, but to employ a ***memory hierarchy***.
- A hierarchy is typically depicted as a ***pyramid***, where the height of a level represents that level's status and width of a level represents the quantity of items at that level relative to the whole.





Locality of Reference

Temporal and Spatial Locality

Temporal Locality

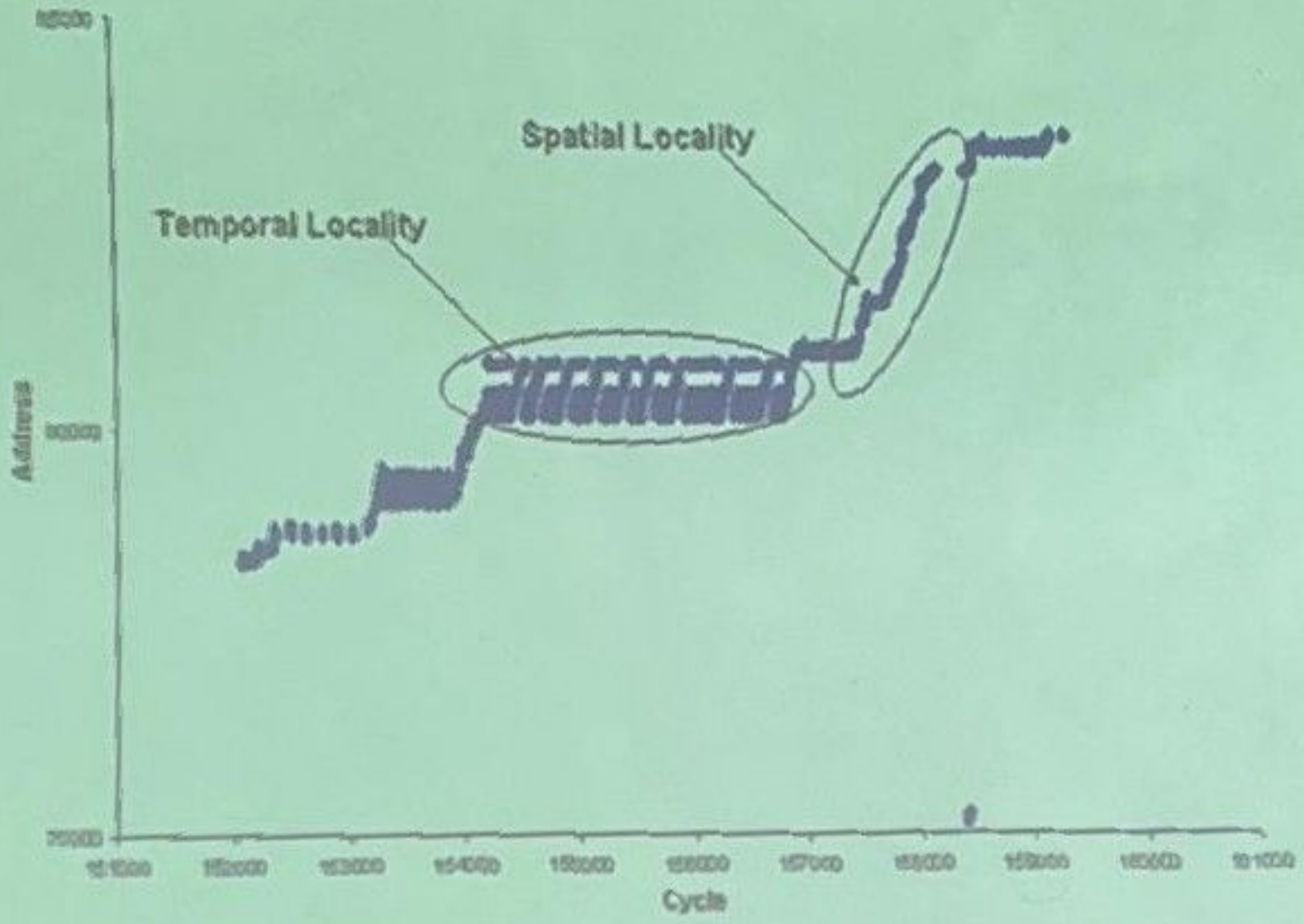
It is a concept that states that *if a resource is accessed at some point of time, then most likely it will be accessed again in a short period of time.*

Spatial Locality

- It is a concept that states that *if a resource is accessed at some point of time, then most likely similar resources will be accessed again in the near future.*
- Sequential locality, a special case of spatial locality, occurs when data elements are arranged and accessed linearly, such as traversing the elements in a one-dimensional array.

Real Life Examples- Cooking in Kitchen, Holiday Planning, Reading Books for an Examination.

Temporal and Spatial Locality



Locality of Reference

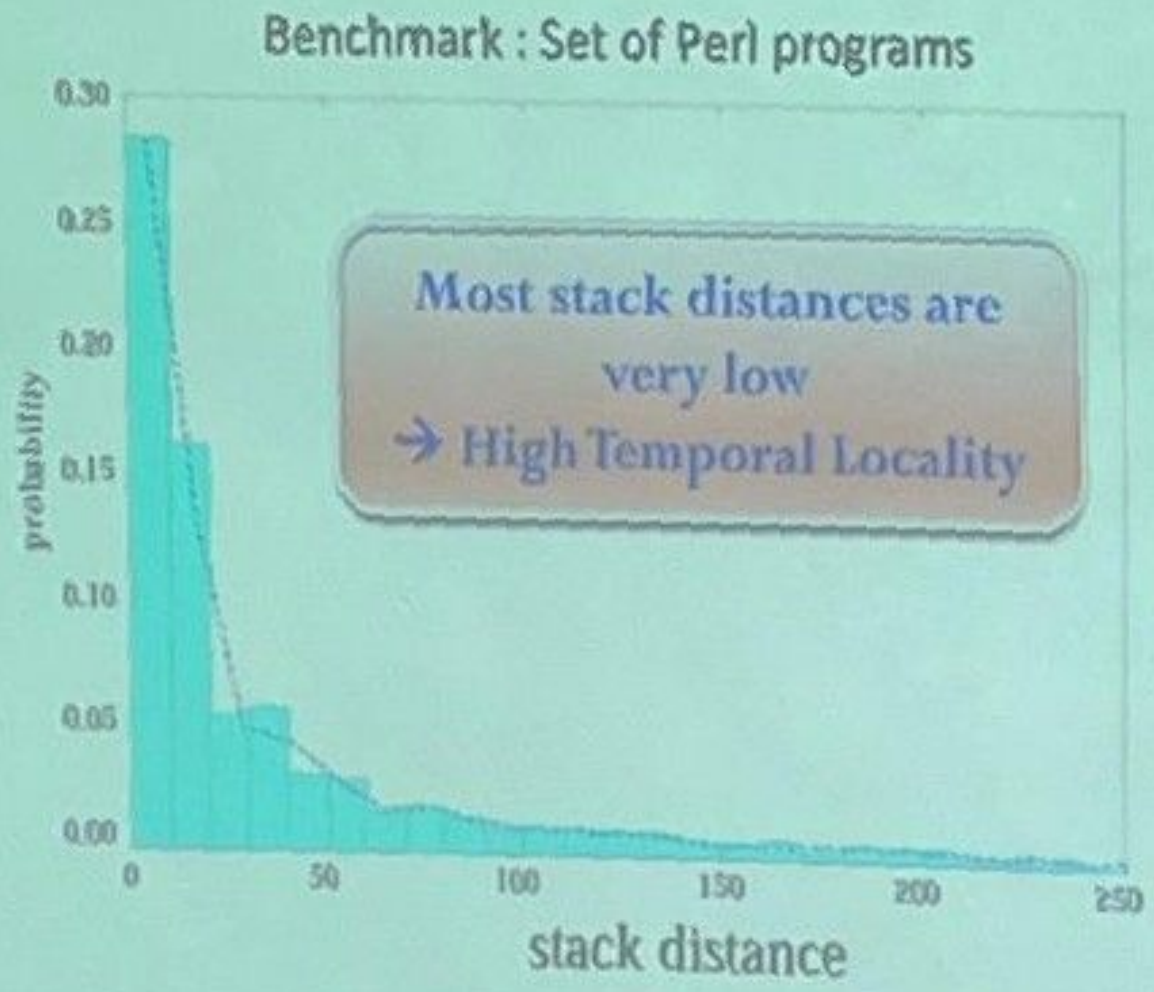
- ***Definition:*** *locality of reference, also known as the principle of locality is the tendency of a processor to access the same set of memory locations repetitively over a period of time.*

- Due to the nature of programming, instructions and data tend to cluster together (loops, subroutines, and data structures)
  - Over a long period of time, clusters will change
  - Over a short period, clusters will tend to be the same

Temporal Locality in Programs

- Let us verify if programs have temporal locality
- Stack distance
  - Have a stack to store memory addresses.
  - Whenever, we access an address → we bring it to the top of the stack
  - Stack distance → Distance from the top of the stack to where the element was found
  - Quantifies reuse of addresses

Stack Distance Distribution



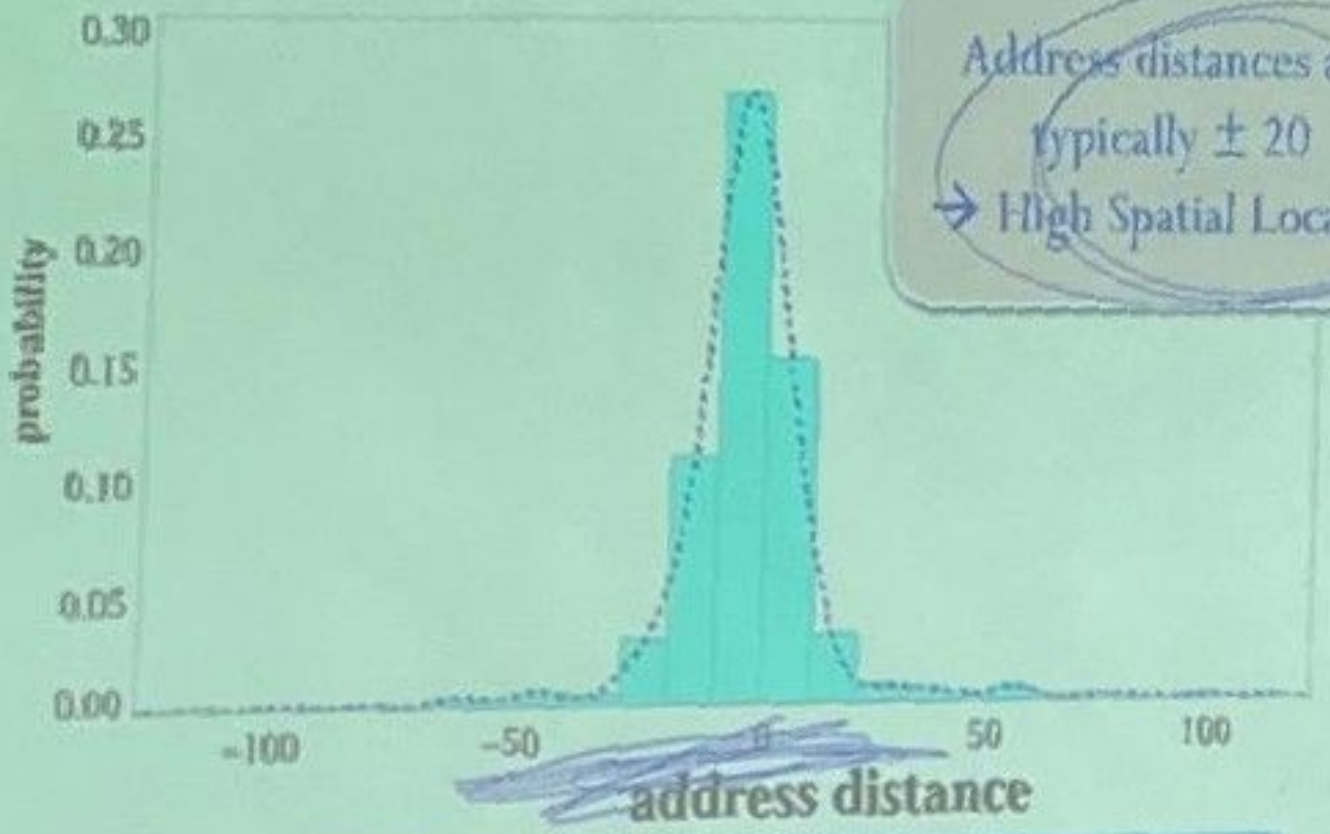
Most stack distances are very low

→ High Temporal Locality

Spatial Locality in Programs
Address Distance

- Maintain a sliding window of the last *K* memory accesses
- Address distance :
  - The ith address distance is the difference in the memory addresses of the ith memory access, and the closest address in the set of last K memory accesses.
  - Shows the **similarity** in addresses

Address Distance Distribution



Address distances are typically ± 20
→ High Spatial Locality

K=10, benchmark consisting of *Perl* programs

Exercise

- Examine the following code. *Identify how many times the processor* **"touches"** *each piece of data and each line of code:*

```
int values[8] =
    {9, 34, 23, 67, 23, 7, 3, 65};
int count;
int sum = 0;
for (count = 0; count < 8; count++)
    sum += values[count];
```

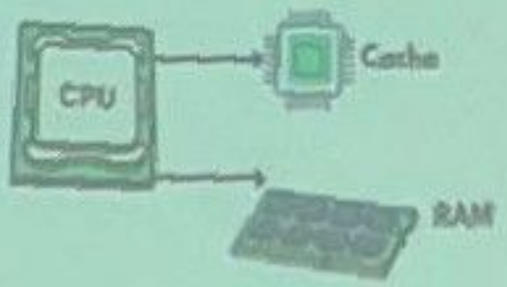
Cache Basics

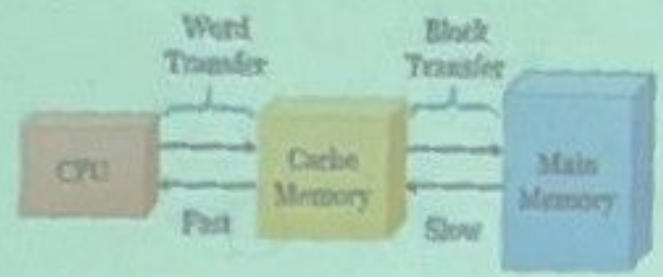
- What is it?
- What makes small fast?
- What is intended?

Cache Basics

- **Cache**
  - *Is a small size (capacity) of fast memory*
  - *Sits between main memory and CPU*
  - *May be located on CPU chip or module*

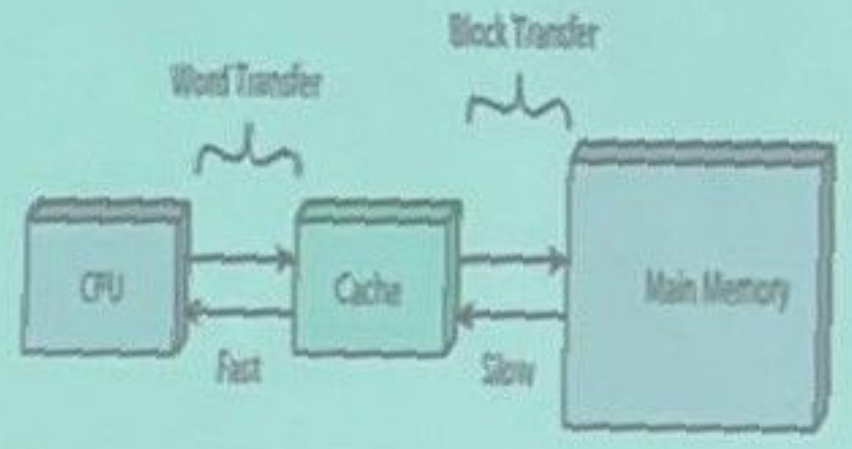
- Cache memory is intended
  - To give memory speed approaching that of the fastest memories available, and
  - At the same time provide a large memory size at the price of less expensive types of semiconductor memories.

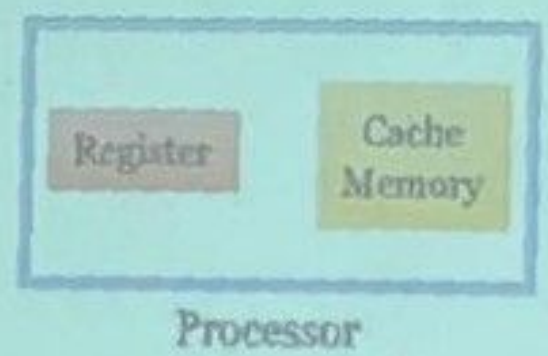




Cache Basics

- What makes small fast?
  - Simpler decoding logic
  - More expensive SRAM technology
  - Close proximity to processor – Cache sits between normal main memory and CPU or it may be located on CPU chip or module



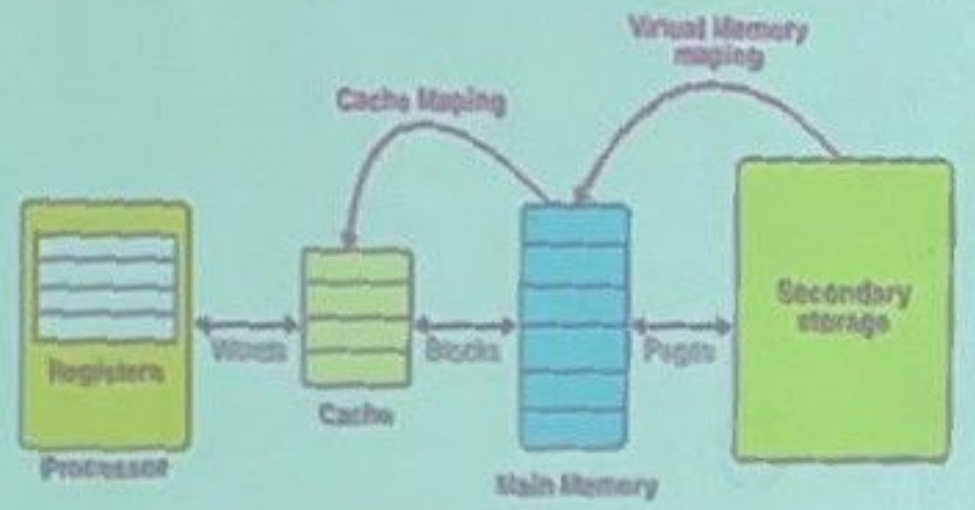


Basic Cache Operations

- lookup → Check if the memory location is present
- data read → read data from the cache
- data write → write data to the cache
- insert → insert a block into a
- replace → find a candidate for replacement
- evict → throw a block out of the cache

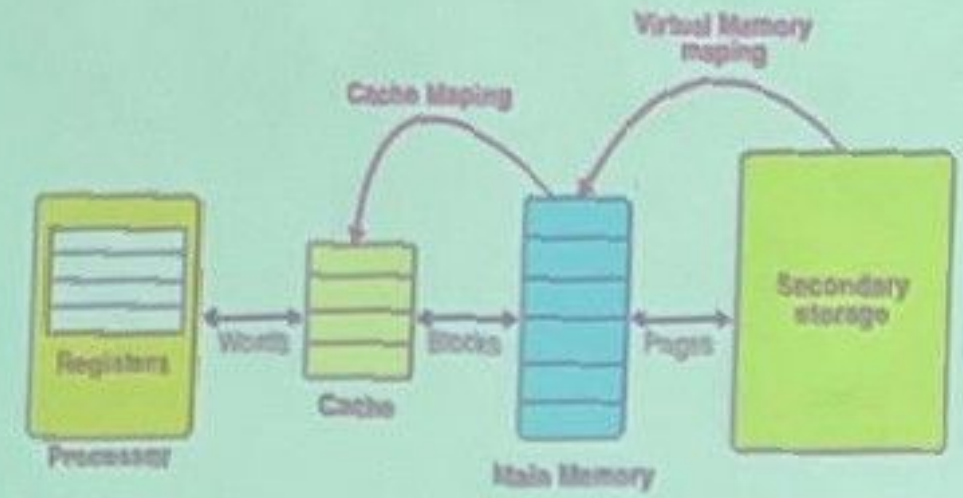
Cache Operations

- CPU requests contents of memory location.
- Check cache for this data- when the processor attempts to read a word of memory, a check is made to determine if the word is in the cache.
- If present, get from cache (fast)
  - The cache contains a copy of portions of main memory.
  - If so, the word is delivered to the processor, i.e., delivered from cache to CPU.

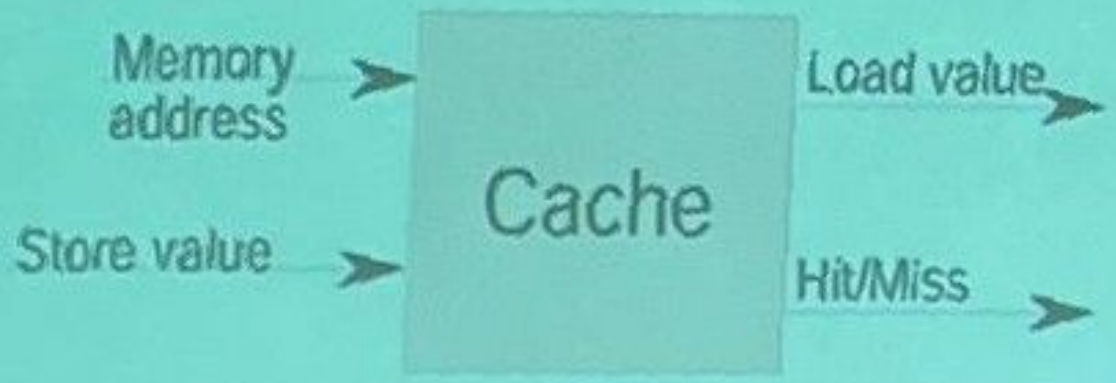


Cache Operations

- If not present, read required block from main memory to cache
  - A block of main memory, consisting of some fixed number of words, is read into the cache and then the word is delivered to the processor.
- Because of the phenomenon of locality of reference, when a block of data is fetched into the cache to satisfy a single memory reference, it is likely that there will be future references to that same memory location or to other words in the block.

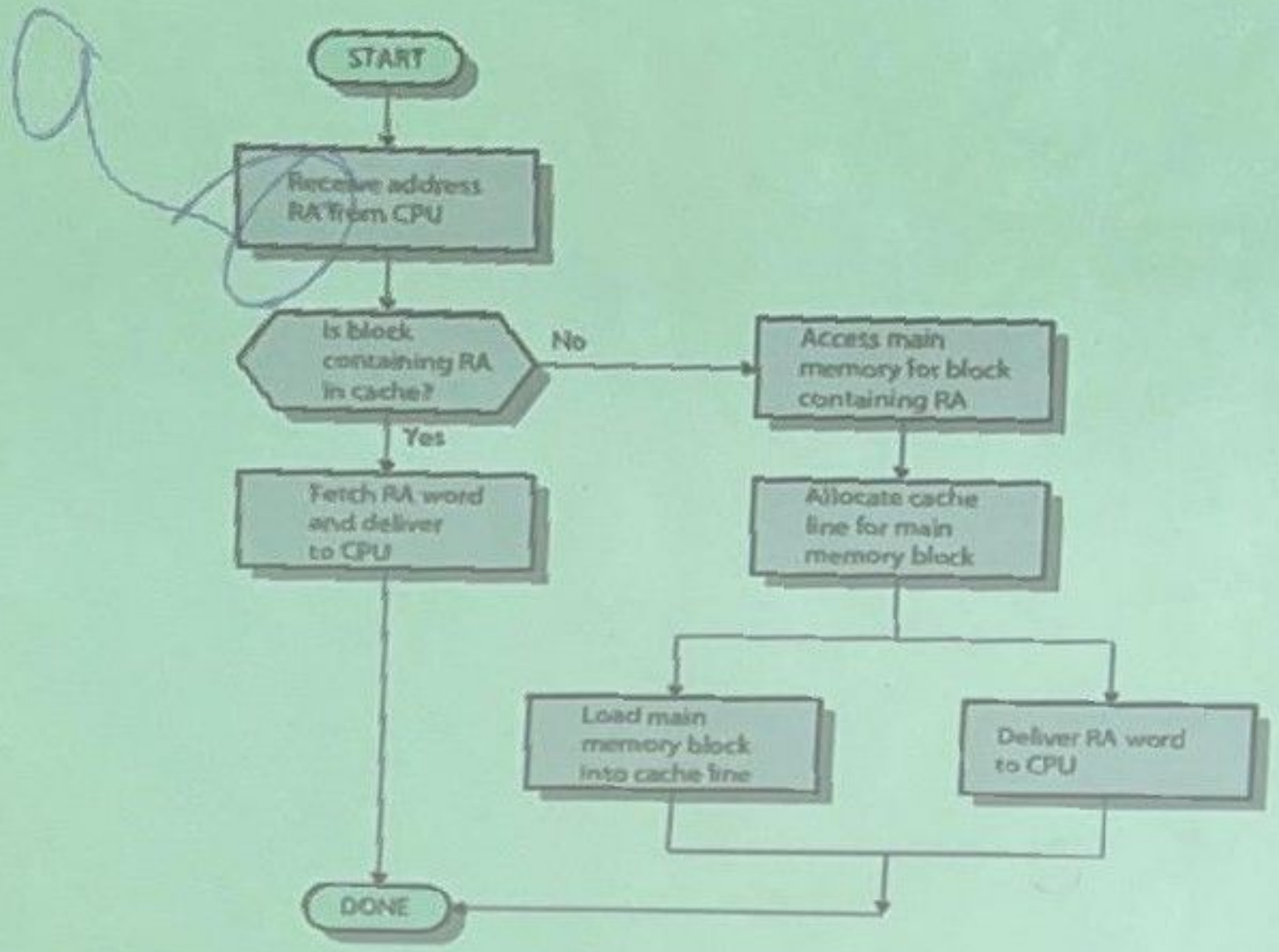


Cache Operations

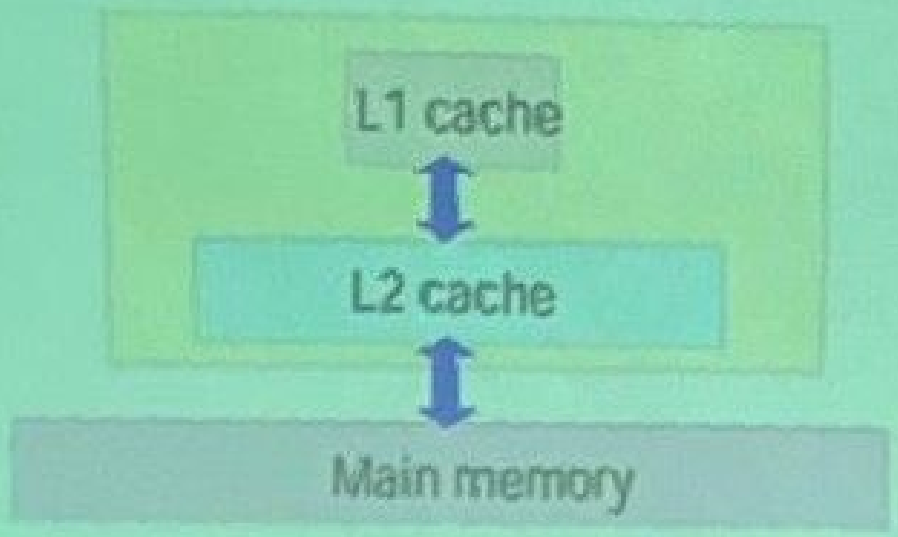


- Saves a subset of memory values
  - We can either have hit or miss
  - The load/store is successful if we have a hit

Cache Read Operation - Flowchart

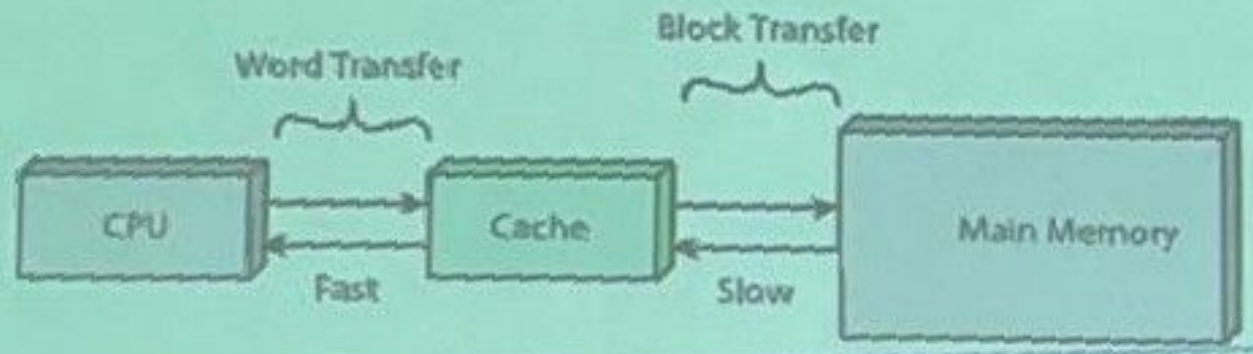


Cache Hierarchy

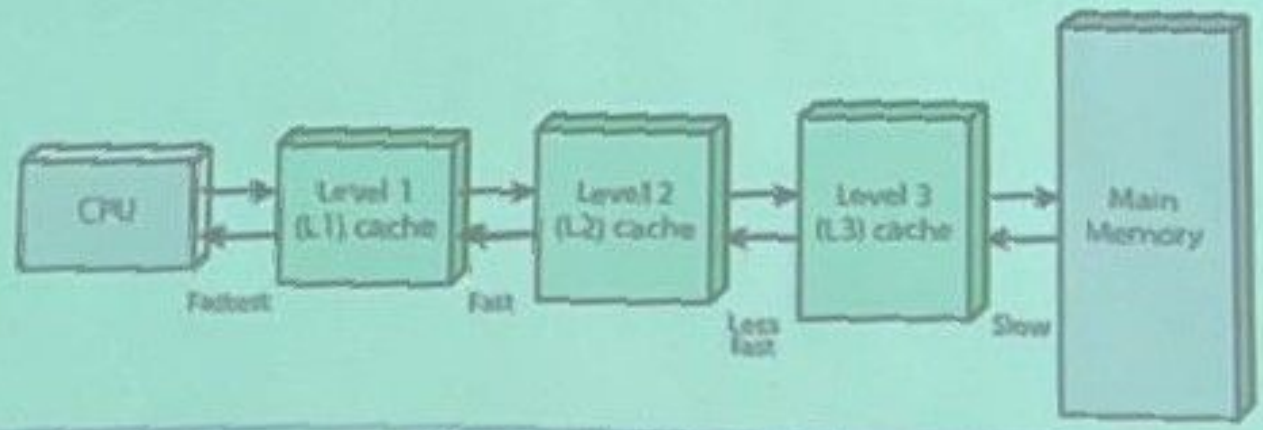


- Cache uses a hierarchical memory system
- L1 (SRAM cells), L2 (SRAM cells), Main Memory (DRAM cells)

Cache Organization Diagram



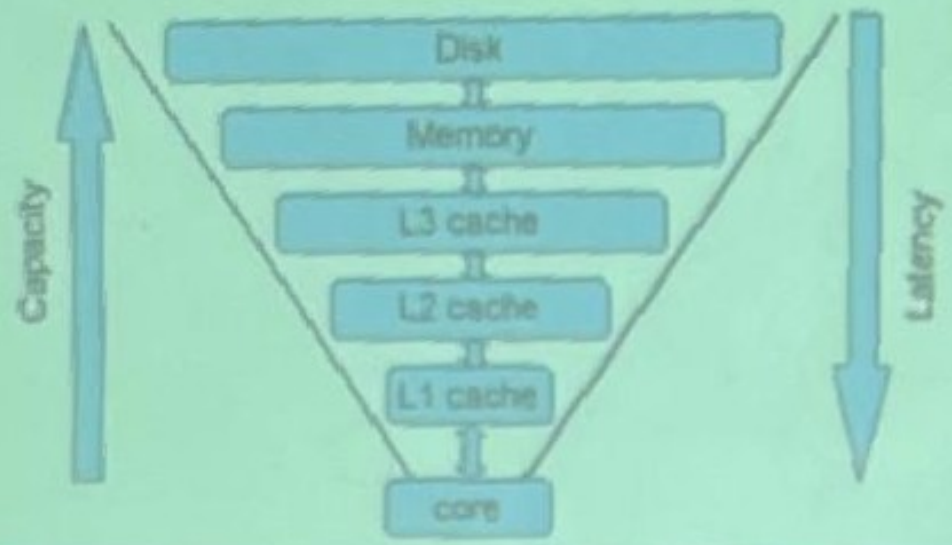
(a) Single cache



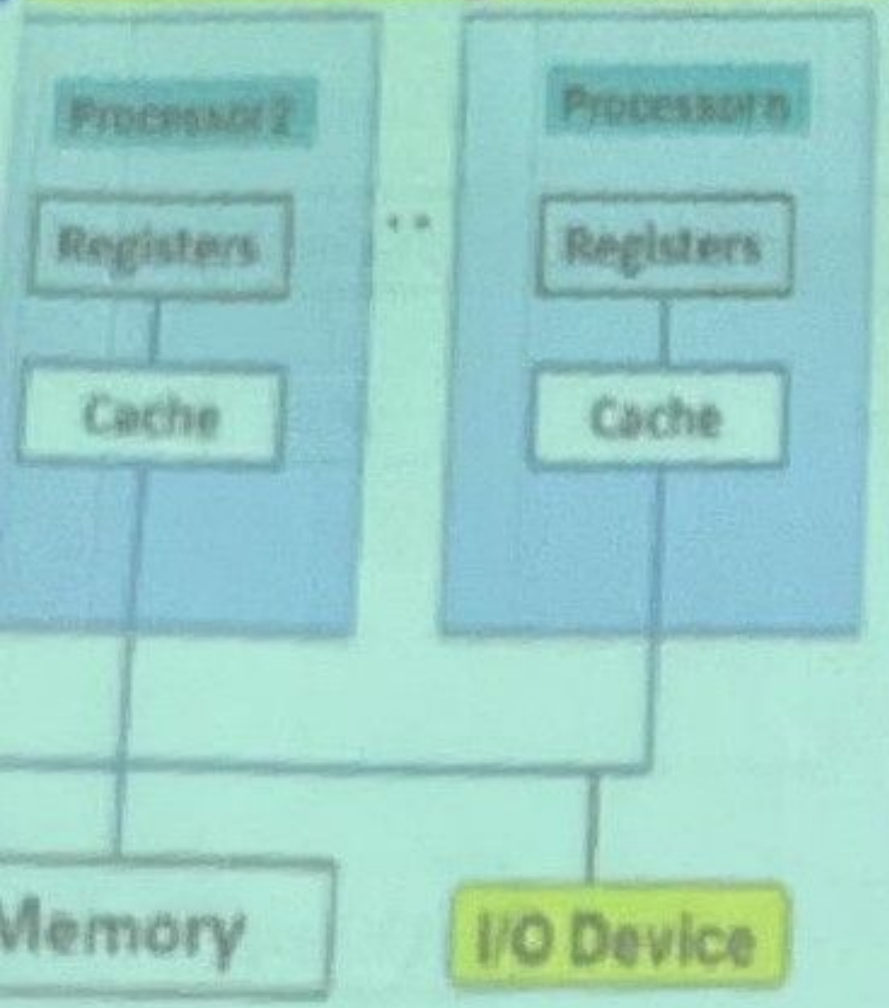
(b) Three-level cache organization

Cache Hierarchy

- The L2 cache is slower and typically larger than the L1 cache, and the L3 cache is slower and typically larger than the L2 cache.
- The L1 cache has size 8-64 KB and is composed of SRAM cells.
- The L2 cache has size 128 KB – 4 MB or more, and is also made of SRAM cells.
- The main memory is much larger with size 1 – 64 GB and made of DRAM cells.



essors Systems



- It stores information as
- Transistors are used to stor
- No refreshing is required.
- It does not have refresh
- These are expensive.
- SRAMs are low density
- Bits are stored in volta
- The technology is used

SRAM Technology

- It stores information as long as the power is supplied.
- *Transistors are used to store information in SRAM.*
- *No refreshing is required.*
- It does not have refreshing unit.
- These are expensive.
- SRAMs are low density devices.
- Bits are stored in voltage form.
- The technology is used in cache memories.

DRAM Technology

- It stores information as long as the power is supplied or a few milliseconds when power is switched off.
- *Capacitors are used to store data in DRAM.*
- *To store information for a longer time, contents of the capacitor needs to be refreshed periodically.*
- It has a refreshing unit.
- DRAM provides slow access speeds.
- These are cheaper.
- DRAMs are high density devices.
- Bits are stored in the form of electric energy.
- These are used in main memories.

SRAM vs DRAM Technology

- **SRAM**
  - **Advantage:** Low power consumption and faster access speeds.
  - **Disadvantage:** Less memory capacities and high costs of manufacturing.

- **DRAM**
  - **Advantage:** Low costs of manufacturing and greater memory capacities.
  - **Disadvantage:** Slow access speed and high power consumption.

Elements of Cache Design

- Size
  - Cache Size,
  - Block Size
  - Line Size
- Addressing
  - Logical
  - Physical
- Mapping Function
  - Direct,
  - Associative,
  - Set Associative
- Write Policy
  - Write through,
  - Write back,
  - Write once
- Number of Caches
  - Single or Multi level,
  - Unified or Split
- Replacement Algorithms
  - Least recently used (LRU),
  - First in first out (FIFO),
  - Least frequently used (LFU),
  - Random